Integration Methods in CppODE

Simon Beyer

2026-05-03

Contents

1 Scope

This vignette describes the four time-stepping methods exposed by `CppODE()`. The methods share a common solver framework (adaptive step-size control, dense Hermite output, sparse and dense LU factorisation, time- and root-triggered events, and forward-mode sensitivity propagation via dual numbers) and differ only in how the individual time step is taken. The Butcher tableaus and numerical coefficients of each method are reproduced in Appendices A–D.

We consider the autonomous initial-value problem

$$\dot{x} = f(x, p_{\mathrm{dyn}}), \qquad x(t_0) = p_{\mathrm{init}}, \tag{1}$$

with state $x \in \mathbb{R}^{n_x}$, parameter vector $p = (p_{\mathrm{init}}, p_{\mathrm{dyn}})$. The Jacobian of the right-hand side will be denoted $J = \partial f / \partial x$ throughout.

2 Stiffness and the choice of method

A linear test problem $\dot{y} = \lambda y$ with $\lambda \in \mathbb{C}^-$ is the standard vehicle for analysing stability. Substituting the test equation into a one-step method gives an amplification factor $R(z)$ with $z = h\lambda$; the method is called *A-stable* if $|R(z)| \le 1$ for all z with $\mathrm{Re}(z) \le 0$, and *A(α)-stable* if the same bound holds in the wedge $\{z \in \mathbb{C} : |\arg(-z)| \le \alpha\}$. *L-stability* adds the requirement $R(z) \to 0$ as $\mathrm{Re}(z) \to -\infty$, which damps stiff transients in a single step.

The initial-value problem (1) is called *stiff* on $[t_0, t_{\mathrm{end}}]$ when an explicit method is forced to take step sizes much smaller than what local accuracy alone would demand, in order to keep the dimensionless products $h\lambda_k$ inside the stability domain, where $\{\lambda_k\}$ are the eigenvalues of J along the trajectory. Stiffness is therefore not a property of the system in isolation; it depends on the spectrum of J, on the integration horizon, and on the requested tolerance. Chemical-kinetics networks with rate constants spanning several orders of magnitude are stiff in this sense, as are many discretised parabolic PDEs.

Stiff problems require methods whose stability domain contains the entire left half-plane (or a wide wedge thereof). Of the four methods listed above, BDF/NDF and Rosenbrock4 satisfy this requirement; Adams-Moulton and Tsit5 do not, and will perform poorly on stiff systems regardless of the requested tolerance.

3 Multistep methods

Linear multistep methods compute x_n from a linear combination of the past $k+1$ state and right-hand-side values $\{x_{n-j}, f_{n-j}\}_{j=0}^{k}$. CppODE implements both the BDF and Adams families through a unified Nordsieck representation, in which the $k+1$ past samples are replaced by the scaled derivative array

$$z_n = \left(x_n,\ h_n \dot{x}_n,\ \frac{h_n^2}{2!}\ddot{x}_n,\ \ldots,\ \frac{h_n^k}{k!} x_n^{(k)} \right),$$

following [1]. Variable-order and variable-step control then reduce to linear transformations of z_n.

3.1 Backward Differentiation Formula (`bdf`)

The BDF formula of order q defines x_n implicitly by the condition

$$\sum_{j=0}^{q} \alpha_j \, x_{n-j} \ = \ h_n \, f(x_n, t_n), \tag{2}$$

which is exact whenever $x(t)$ is a polynomial of degree at most q on the past $q+1$ nodes. The method is A-stable for $q \le 2$ and A(α)-stable for $q \in \{3, 4, 5\}$; the stability angle α shrinks monotonically from 86° at $q = 3$ to 51° at $q = 5$.

The Newton iteration for the corrector (2) solves

$$W\Delta x^{(\nu)} = -\tilde{r}\left(x_n^{(\nu)}\right), \qquad x_n^{(\nu+1)} = x_n^{(\nu)} + \Delta x^{(\nu)}, \tag{3}$$

at each iterate ν, with leading coefficient $\beta_0 = 1/\alpha_0$ and a residual $\tilde{r}$ obtained by dividing the corrector residual by the scalar $h_n\beta_0$. Following the Hindmarsh/SUNDIALS scaling convention, CppODE defines the iteration matrix in scaled form,

$$W = \frac{1}{h_n\,\beta_0}\mathbb{1} - J(x_n^{(\nu)}, t_n), \tag{4}$$

which is the textbook iteration matrix $I - h_n\beta_0\,J$ divided by the scalar factor $h_n\beta_0$. The same scaled form is adopted for Rosenbrock4 in (8) below (with β_0 replaced by the diagonal coefficient γ), and both methods share a single `factorize_W` implementation in `cppode_lu.hpp` that materialises $-J$ via codegen and adds $1/(h_n\beta_0)$ (resp. $1/(\gamma h_n)$) to the diagonal before factorisation. The scaled form admits an $O(n_x)$ diagonal-only refactorisation when only h_n changes, and it leaves a clean $-J$ in cache for the implicit-function-theorem step needed by the AD-aware LU. The factorisation of W is the dominant cost per step and is reused for as long as the contraction rate of the Newton iteration permits.

CppODE replaces classical BDF by the Numerical Differentiation Formulas (NDF) of [2] by default. The NDF modification adds a correction proportional to the Nordsieck predictor error,

$$\sum_{m=1}^{q}\frac{1}{m}\,\nabla^m x_n - h_n f(x_n, t_n) - \kappa_q\,\gamma_q\,\left(x_n - x_n^{\text{pred}}\right) = 0, \qquad \gamma_q = \sum_{m=1}^{q}\frac{1}{m}, \tag{5}$$

where ∇ denotes the backward-difference operator. The algebraic effect is to scale β_0 in (3) by $(1-\kappa_q)$ and to multiply the leading truncation-error coefficient $1/(q+1)$ by $(1+\kappa_q(q+1)\gamma_q)$. Choosing $\kappa_q < 0$ shrinks the truncation-error coefficient without disturbing the iteration matrix, allowing larger step sizes at the same local error. The numerical κ_q values adopted by CppODE are tabulated in Appendix A; empirically they yield a 25 % step-size gain over classical BDF for $q \le 4$, at the price of a few degrees of stability angle. Setting `useNDF = FALSE` recovers the classical BDF formulas ($\kappa_q \equiv 0$).

A typical accepted step performs one right-hand-side evaluation for the predictor and between one and three right-hand-side evaluations together with the same number of linear solves for the corrector. A new Jacobian evaluation and LU factorisation are triggered only when the Newton iteration stalls or when the step size or order changes substantially. BDF/NDF is the default method in CppODE and the recommended choice when stiffness is suspected.

3.2 Adams-Moulton in PECE form (`adams`)

The Adams-Moulton family integrates the local interpolant of f over $[t_{n-1}, t_n]$. Writing f as the unique polynomial through the $q+1$ samples $\{(t_{n-j}, f_{n-j})\}_{j=0}^{q}$ and integrating gives

$$x_n \;=\; x_{n-1} \;+\; h_n \sum_{j=0}^{q}\beta_j\, f(x_{n-j}, t_{n-j}), \tag{6}$$

which is implicit in x_n through the $j = 0$ term. CppODE handles the implicitness in predictor-evaluate-corrector-evaluate (PECE) mode: an explicit predictor (Adams-Bashforth of order q) supplies x_n^P, the right-hand side is evaluated once at x_n^P, the corrector (6) is then evaluated explicitly using $f(x_n^P, t_n)$ in place of $f(x_n, t_n)$, and a final right-hand-side evaluation produces the value entered into the Nordsieck history. No Newton iteration and no linear solve are performed.

The Adams-Moulton stability domain on the negative real axis shrinks rapidly with order; for $q \ge 3$ it does not contain even a small wedge of $\mathbb{C}^-$ away from the origin, so the method is unsuitable for stiff problems. For non-stiff problems with smooth solutions and a long integration horizon, however, the high attainable order ($q_{\max} = 12$) and the cost of two right-hand-side evaluations per accepted step make it competitive with explicit Runge-Kutta methods of fixed order.

4 Single-step methods

Single-step methods carry no information beyond the current state. They achieve order $p > 1$ by evaluating internal stages within each step rather than by reaching back into the integration history. The absence of history makes restarts after events trivial; both methods below restart at full order in a single step, unlike the multistep methods, which must rebuild the Nordsieck history at order 1.

4.1 Rosenbrock (`rb4`)

A Rosenbrock method of s stages with diagonal coefficient γ linearises the implicit Runge-Kutta condition about (x_n, t_n) and defines stage values k_i by

$$W k_i = f\left(x_n + \sum_{j<i} \alpha_{ij}\, k_j,\; t_n + c_i\, h_n\right) + \frac{1}{h_n} \sum_{j<i} \frac{\gamma_{ij}}{\gamma}\, k_j + h_n\, d_i\, \frac{\partial f}{\partial t}(x_n, t_n), \tag{7}$$

with the iteration matrix

$$W = \frac{1}{\gamma\, h_n} \mathbb{1} - J(x_n, t_n). \tag{8}$$

The method coefficients are listed in Table 1.

Symbol	Meaning
α_{ij}	stage combination weights (strict lower triangular)
c_i	stage time abscissae, $c_i = \sum_{j<i} \alpha_{ij}$
γ_{ij}	coupling coefficients (strict lower triangular)
γ	diagonal coefficient; governs L-stability
d_i	weights for the autonomous extension ($\partial f/\partial t$)
m_i	quadrature weights for the order-4 solution
$\hat{m}_i$	quadrature weights for the embedded order-3 error estimate

Table 1: Rosenbrock method coefficients.

The advanced solution and embedded error estimate are

$$x_{n+1} = x_n + \sum_{i=1}^{s} m_i\, k_i, \qquad \hat{x}_{n+1} \;=\; x_n + \sum_{i=1}^{s} \hat{m}_i\, k_i.$$

Each stage (7) is a single linear solve; no Newton iteration is required. Per accepted step CppODE evaluates J and factorises W once, performs five right-hand-side evaluations, and back-substitutes six times.

CppODE uses the order-4 coefficients of [3, Sec. IV.7], the same set as the `rosenbrock4` stepper of Boost.Numeric.Odeint. The pair is L-stable and carries an embedded order-3 estimator. Continuous output on $[t_n, t_{n+1}]$ is delivered by an order-3 interpolant constructed from the stage values k_i and two additional coefficient sets $d_i^{(2)}$ and $d_i^{(3)}$ tabulated in Appendix C; no extra right-hand-side evaluations are required for output points.

Rosenbrock methods are competitive with BDF on stiff problems whose Jacobian is cheap relative to the right-hand side, and they avoid the multistep restart cost after events. They are therefore the recommended stiff method for systems with frequent state discontinuities on medium sized problems.

4.2 Tsitouras (`tsit5`)

Tsitouras is the explicit Runge-Kutta pair of order 5(4) derived by [4] under the simplifying assumption that the first column of the coefficient matrix vanishes, $a_{i,1} = 0$ for $i = 2, \ldots, s$. Removing this column reduces the number of order conditions to be satisfied during construction and was shown to admit a

particularly favourable choice of nodes c_i in terms of leading-error norm. The pair has $s = 7$ stages with the First-Same-As-Last (FSAL) property,

$$k_7 \;=\; f(x_{n+1},\, t_n + h_n) \;=\; k_1 \text{ at step } n+1,$$

so an accepted step costs six right-hand-side evaluations rather than seven. The method coefficients are listed in Table 2.

Symbol	Meaning
a_{ij}	Runge-Kutta stage weights (strict lower triangular, first column zero)
c_i	stage time abscissae, $c_i = \sum_{j<i} a_{ij}$
b_i	quadrature weights for the order-5 solution; $b_7 = 0$
$\hat{b}_i$	quadrature weights for the embedded order-4 estimate; $\hat{b}_7 \neq 0$
k_i	stage derivatives, $k_i = f\Big(x_n + h_n \sum_{j<i} a_{ij} k_j,\; t_n + c_i h_n\Big)$

Table 2: Tsit5 method coefficients.

The advanced solution and embedded error estimate are

$$x_{n+1} = x_n + h_n \sum_{i=1}^{7} b_i\, k_i, \qquad \hat{x}_{n+1} = x_n + h_n \sum_{i=1}^{7} \hat{b}_i\, k_i,$$

and the local error estimate used for step-size control is

$$e_{n+1} = h_n \sum_{i=1}^{7} (\hat{b}_i - b_i)\, k_i.$$

Continuous output is produced by a cubic Hermite interpolant on $[t_n, t_{n+1}]$, using the accepted-step endpoints (x_n, x_{n+1}) and their derivatives $(f_n, f_{n+1}) = (k_1, k_7)$, both already in cache. No additional right-hand-side evaluations are required for output points; the interpolation order ($p = 4$ globally, $p = 3$ locally on the interval) is one less than the order of the underlying step.

Tsitouras has no Jacobian and no linear solve and performs the largest amount of integration per right-hand-side call of the four methods considered here. It is the appropriate choice for non-stiff problems whose right-hand side is expensive to evaluate.

5 Common solver infrastructure

The following machinery is shared across all four methods.

5.1 Adaptive step-size control

The error estimate e_n delivered by each method is reduced to a scalar through the weighted root-mean-square norm

$$\|e_n\|_{\mathrm{WRMS}} = \sqrt{\frac{1}{n_x} \sum_{j=1}^{n_x} \left(\frac{e_{n,j}}{w_{n,j}} \right)^2}, \qquad w_{n,j} = \texttt{atol} + \texttt{rtol}\, |x^{\mathrm{ref}}_{n,j}|, \tag{9}$$

in which $w_{n,j}$ is a per-component tolerance combining an absolute floor `atol` and a relative scale `rtol` $|x^{\mathrm{ref}}_{n,j}|$. Dividing $e_{n,j}$ by $w_{n,j}$ before squaring puts components of vastly different magnitudes on equal footing and removes the choice of physical units from the acceptance test. The reference state x^{ref}_n from which the relative scale is computed is the Nordsieck predictor in BDF/NDF and the component-wise maximum $\max(|x_n|, |x_{n+1}|)$ in the one-step methods; both conventions are standard and rarely alter the norm appreciably. A step is accepted whenever $\|e_n\|_{\mathrm{WRMS}} \leq 1$.

The proposed next step size is set by an order-aware controller of the type analysed in [3],

$$h_{n+1} = h_n \min\left\{\rho_{\max}, \max\left(\rho_{\min}, S\,\|e_n\|_{\mathrm{WRMS}}^{-1/(p+1)}\|e_{n-1}\|_{\mathrm{WRMS}}^{\beta/(p+1)}\right)\right\},$$

with safety factor $S < 1$, growth limits $\rho_{\min}, \rho_{\max}$, and PI-filter coefficient $\beta \in [0, 1]$. The multistep methods use $\beta = 0$ (I-controller); the single-step methods use $\beta > 0$ to damp high-frequency oscillations of h_n, following the PI control of [3].

5.2 Dense output

Output requested at user-specified time points falling between two accepted steps is served by Hermite interpolation that consumes only information already produced for the step. No additional right-hand-side evaluations are performed for dense output. The local interpolation order matches the order of the step (order 4 for Rosenbrock4 and Tsit5; the multistep methods reuse the Nordsieck history, which carries derivatives up to order q).

5.3 Automatic Forward sensitivities

Dual-number arithmetic

Forward-mode automatic differentiation propagates derivative information alongside the primal computation without forming finite differences or symbolic expressions. The fundamental object is a *dual number*

$$\tilde{x} = x + x'\,\varepsilon, \qquad \varepsilon^2 = 0, \quad \varepsilon \neq 0,$$

where $x \in \mathbb{R}$ is the primal value and $x' \in \mathbb{R}$ is its directional derivative with respect to some chosen seed direction. Arithmetic on dual numbers follows from the truncated Taylor expansion: for two dual numbers $\tilde{x} = x + x'\varepsilon$ and $\tilde{y} = y + y'\varepsilon$,

$$\tilde{x} + \tilde{y} = (x + y) + (x' + y')\varepsilon, \qquad \tilde{x} \cdot \tilde{y} = xy + (xy' + x'y)\varepsilon,$$

and for a smooth scalar function g,

$$g(\tilde{x}) = g(x) + g'(x)\,x'\,\varepsilon.$$

The chain rule is therefore implemented *by construction*: any composition of dual-number operations automatically tracks the derivative of the composed function. No code transformation or symbolic manipulation is required; the same source code that evaluates f over $\mathbb{R}$ evaluates both f and $\partial f/\partial x$ when called over the dual algebra.

Sensitivity equations via dual evaluation

Let the ODE right-hand side depend on a parameter vector $p \in \mathbb{R}^{n_p}$,

$$\dot{x} = f(x, p, t), \qquad x(t_0) = x_0(p),$$

and define the *sensitivity matrix*

$$S_p = \frac{\partial x}{\partial p} \in \mathbb{R}^{n_x \times n_p}, \qquad (S_p)_{ij} = \frac{\partial x_i}{\partial p_j}.$$

To derive the evolution equation for S_p via dual arithmetic, seed each state component with its row of S_p and form the dual-valued state vector

$$\tilde{x}_i = x_i + (S_p)_{ij}\,\varepsilon_j,$$

where ε_j is the dual unit associated with direction p_j and summation over j is implied. Evaluating f component-wise over the dual algebra and collecting the ε_j-coefficient of the i-th output gives

$$\left[f(\tilde{x}, p, t)\right]_{i,\,\varepsilon_j} = \frac{\partial f_i}{\partial x_k}\,(S_p)_{kj} + \frac{\partial f_i}{\partial p_j},$$

where repeated k is summed. Since the primal part of the dual evaluation recovers $\dot{x}_i = f_i(x, p, t)$, differentiating $\dot{x}_i = f_i$ with respect to p_j and identifying the ε_j-coefficient with $(\dot{S}_p)_{ij}$ yields

$$\dot{S}_p = \frac{\partial f}{\partial x}\, S_p + \frac{\partial f}{\partial p}, \qquad S_p(t_0) = \frac{\partial x_0}{\partial p}. \tag{10}$$

Hence a single dual-number evaluation of f propagates the full sensitivity matrix S_p in lockstep with the primal state x, at a cost proportional to n_p. No additional Jacobian evaluations are required; the same code path that computes f computes $(\partial f/\partial x)\, S_p + \partial f/\partial p$ automatically.

Seeding: from S_p to S_θ

In practice the solver is differentiated not with respect to the raw parameter vector p but with respect to a higher-level vector $\theta \in \mathbb{R}^M$ that influences both the dynamic parameters p_{dyn} and the initial condition $x_0 = p_{\text{init}}(\theta)$. Define

$$S_\theta = \frac{\partial x}{\partial \theta} \in \mathbb{R}^{n_x \times M}.$$

By the chain rule, $S_\theta = S_p\, \partial p/\partial\theta$, and substituting into (10) gives

$$\dot{S}_\theta = \frac{\partial f}{\partial x}\, S_\theta + \frac{\partial f}{\partial p_{\text{dyn}}}\, \frac{\partial p}{\partial \theta}, \qquad S_\theta(t_0) = \frac{\partial p_{\text{init}}}{\partial \theta}. \tag{11}$$

The matrix $\partial p/\partial\theta$ is the *seed matrix*, supplied by the user and fixed for the duration of the integration. Choosing $\partial p/\partial\theta = I_{n_p}$ recovers $S_\theta = S_p$; a non-trivial seed allows the sensitivity computation to be carried out in any M directions of interest.

Dual-number aware error norm

When forward sensitivities are active, the integrator carries the sensitivity matrix $S_\theta \in \mathbb{R}^{n_x \times M}$ alongside the primal state. Every accepted step then produces $1 + M$ error vectors of length n_x: the primal estimate $e_n^{(x)} \in \mathbb{R}^{n_x}$ and a sensitivity estimate $e_S^{(k)} \in \mathbb{R}^{n_x}$ for each column $\partial x/\partial\theta_k$ of S_θ. Two natural ways to combine these into a single scalar suggest themselves: a flat WRMS that pools all $n_x(1+M)$ components into one sum, or a maximum over per-vector WRMS norms. CppODE adopts the second, following the staggered-tolerance convention of CVODES (the `CV_STAGGERED` mode of its sensitivity solver).

The state norm $\|e_n^{(x)}\|_{\text{WRMS}}$ is exactly (9). The sensitivity column norm is its analogue, with S_θ replacing x in both the error vector and the weight,

$$\|e_S^{(k)}\|_{\text{WRMS}} = \sqrt{\frac{1}{n_x}\sum_{i=1}^{n_x}\left(\frac{(e_S)_{ik}}{w_i^{(k)}}\right)^2}, \qquad w_i^{(k)} = \texttt{atol} + \texttt{rtol}\,|(S_\theta)_{ik}^{\text{ref}}|, \tag{12}$$

in which $(S_\theta)_{ik}^{\text{ref}}$ is the reference value of the i-th component of column k, taken from the same source (predictor or pre/post maximum) as x_n^{ref} for the state. The scalar that the step-size controller sees is the worst of the state norm and the M column norms,

$$\|e_n\|_{\text{WRMS}} = \max\left\{\|e_n^{(x)}\|_{\text{WRMS}},\ \max_{1\le k\le M}\|e_S^{(k)}\|_{\text{WRMS}}\right\}. \tag{13}$$

The choice of the maximum rather than the flat WRMS is deliberate. A flat WRMS divides by the total component count $n_x(1 + M)$ and therefore averages the squared errors across directions in θ: a large error in one sensitivity column k_0 can be diluted by M small errors in the remaining columns and the state, which would let the integrator take an over-aggressive step in the direction k_0. The maximum convention prevents this dilution and forces every direction in θ to satisfy the requested tolerance on its own. Since the controller still operates on a single scalar, the cost is at most a constant overhead in the number of accepted steps, and the control-law parameters $(S, \rho_{\min}, \rho_{\max}, \beta)$ need no adjustment.

The same WRMS structure also governs the Newton convergence test in BDF/NDF: a Newton iterate is accepted only when the iteration correction passes (13), with the maximum again taken over state and per-column sensitivity components. The non-linear iteration therefore converges sensitivity-uniformly: a tight Newton solve in the primal does not allow a loose solve in some sensitivity direction, and a parameter direction that drives the correction up will trigger more Newton iterations or a step-size reduction even when the primal residual is already small.

5.4 Events and restarts

CppODE supports two kinds of events that interrupt smooth integration. A *time-triggered event* fires at a user-specified time $t_e(\theta)$ that may depend on θ through a closed-form expression. A *root-triggered event* fires when a user-supplied event function $g_e(x,t)$ crosses zero along the trajectory; the firing time $t^*(\theta)$ is then defined implicitly by $g_e(x(t^*),t^*)=0$. In both cases an event applies a discontinuous reset to the state,

$$x^+ = g(x^-,t),$$

typical examples being instantaneous dosing of a kinetic species, a clipping rule that enforces non-negativity, or a switch between dynamical regimes. The remainder of this section first describes the cost of restarting the integrator after a discontinuity, then sets up the sensitivity-transport problem, and finally derives the correction for each event type.

Restart cost

Single-step methods (Rosenbrock4, Tsit5) carry no information beyond the current state and resume integration in a single full-order step after the reset. Multistep methods (BDF/NDF, Adams) build their order through history and must restart at order 1, rebuilding the Nordsieck array from scratch over several steps. This restart penalty is the dominant cost on problems with frequent events and is the reason Rosenbrock4 is the recommended stiff method when discontinuities are common, even though BDF/NDF is the default elsewhere.

Sensitivity transport: the geometric picture

The discontinuous map g acts on the trajectory $x(t,\theta)$ at the firing time $t^*(\theta)$. The sensitivity S_θ at the *next grid time* after the event is therefore not simply $(\partial g/\partial x)\,S_\theta^-$ evaluated at the grid time: the firing time itself depends on θ, and shifting it by δt^* also shifts the trajectory by $f^-\,\delta t^*$ on the pre-event side and by $f^+\,\delta t^*$ on the post-event side. Two distinct contributions therefore compose:

1. The pre-event flow carries x^- along $\dot{x}=f^-$ from the grid time $t_*^{(0)} := \mathrm{scalar}(t^*)$ to the θ-perturbed event time t^*, a distance $\delta t^* = t^* - t_*^{(0)}$ that has zero scalar part but non-zero θ-derivatives;
2. The post-event flow carries $g(x^-,t^*)$ along $\dot{x}=f^+$ from t^* back to the next grid time, again over a θ-dependent distance.

If these two shifts are ignored and the sensitivity is updated by the naive expression $S_\theta^+ = (\partial g/\partial x)\,S_\theta^-$, the result is wrong by a term of size $f^\pm\,(\partial t^*/\partial\theta)$ on each side. The correct first-order update absorbs both shifts into the saltation correction discussed at the end of this section.

CppODE realises the correction purely through the dual-number arithmetic of the previous section, augmented by a Heun transport. Every quantity along the way is evaluated as a dual number: the right-hand side f, the event function g_e, the reset map g, and the time arguments themselves. The Heun sandwich therefore transports first-order *and* second-order sensitivities in lockstep with the state. For pure-double integration (no AD), no transport is needed: the reset is applied as a plain map and the two sub-sections below are skipped.

The construction differs in detail between time-triggered and root-triggered events, because the firing-time perturbation δt enters the dual evaluation differently: directly in the time-triggered case, where $t_e(\theta)$ is supplied by the user and its dual derivatives propagate through the chain rule, and implicitly in the root case, where $t^*(\theta)$ must be reconstructed from the condition $g_e=0$ via the implicit function theorem.

Time-triggered events

For an event at $t = t_e(\theta)$, the integrator advances on the scalar grid $t_e^{(0)} := \mathrm{scalar}(t_e)$ and treats the dual residual

$$\delta t = t_e - t_e^{(0)}, \qquad \mathrm{scalar}(\delta t) = 0, \tag{14}$$

as the directional perturbation of the firing time. Its first-order dual components carry $\partial t_e/\partial\theta_a$ and its second-order dual components carry $\partial^2 t_e/\partial\theta_a\partial\theta_b$, both inherited automatically from the user expression for $t_e(\theta)$. The state at $t_e^{(0)}$ is denoted x^- and likewise carries S_θ in its first-order dual slots.

The transport across the event is performed in three sub-steps: a forward Heun shift along $\dot{x} = f$ from $t_e^{(0)}$ to t_e, the reset map g at t_e, and a backward Heun shift from t_e back to $t_e^{(0)}$,

$$\begin{aligned}
&\text{forward:} \quad f_1 = f(x^-, t_e^{(0)}), \quad x_E = x^- + f_1\,\delta t, \quad f_2 = f(x_E, t_e),\\
&\qquad\qquad x^* = x^- + \tfrac{1}{2}(f_1 + f_2)\,\delta t,\\
&\text{reset:} \quad x_+^* = g(x^*, t_e),\\
&\text{backward:} \quad \tilde{f}_1 = f(x_+^*, t_e), \quad x_B = x_+^* - \tilde{f}_1\,\delta t, \quad \tilde{f}_2 = f(x_B, t_e^{(0)}),\\
&\qquad\qquad x^+ = x_+^* - \tfrac{1}{2}(\tilde{f}_1 + \tilde{f}_2)\,\delta t.
\end{aligned} \tag{15}$$

The forward leg evaluates f at both endpoints ($t_e^{(0)}$ and t_e); averaging the two evaluations is the standard Heun (improved Euler) rule and produces a result accurate to second order in δt. The reset g is then applied on the event surface at t_e, and the backward leg is the symmetric Heun shift to the grid time, this time along the post-event flow.

A one-stage Euler shift would suffice for first-order sensitivities. This is because $\mathrm{scalar}(\delta t) = 0$ implies that the first-order dual content of δt^k vanishes for every $k \geq 2$, so any term of order δt^2 or higher contributes nothing to $\partial x^+/\partial\theta$. Once second-order sensitivities are switched on, however, the situation changes: the curvature contribution

$$\tfrac{1}{2}(J\,f + \partial f/\partial t)\,\delta t^2, \qquad J = \partial f/\partial x,$$

acquires a non-vanishing second-order dual component, because the square of δt inherits a non-zero second derivative from the product of two first-derivative slots,

$$\frac{\partial^2(\delta t)^2}{\partial\theta_a\,\partial\theta_b} = 2\,\frac{\partial t_e}{\partial\theta_a}\,\frac{\partial t_e}{\partial\theta_b},$$

even though both its value and its first derivatives are zero. The Heun midpoint absorbs the curvature term implicitly through the second right-hand-side evaluation at the shifted state *and* the shifted time. The time argument in f_2 and $\tilde{f}_2$ is essential: omitting it would lose the $\frac{1}{2}(\partial f/\partial t)\,\delta t^2$ part for systems with explicit time dependence, even though the time shift itself is invisible at the scalar level ($\mathrm{scalar}(\delta t) = 0$).

Root-triggered events

For a root event the firing time $t^*(\theta)$ is defined implicitly by

$$G(t^*(\theta), \theta) := g_e(x(t^*(\theta), \theta), t^*(\theta)) = 0. \tag{16}$$

Three steps lead from this condition to the corrected state at the next grid time: localisation of t^* on the scalar trajectory, reconstruction of the dual residual δt^* via the implicit function theorem, and the same forward-reset-backward Heun transport as in the time-triggered case.

Localisation The integrator detects a root by tracking the sign of g_e at successive accepted steps: when the sign of $g_e(x_n, t_n)$ and $g_e(x_{n+1}, t_{n+1})$ disagree, a sign change must have occurred on $[t_n, t_{n+1}]$. The crossing is then refined by bisection using the dense output of the accepted step, which means no extra right-hand-side evaluations are spent on the localisation itself. The bisection terminates when the bracket width falls below a user-supplied root tolerance or below an integrator-determined floor; the midpoint of the final bracket is returned as $t_*^{(0)} := \mathrm{scalar}(t^*)$. The dual residual $\delta t^* = t^* - t_*^{(0)}$ must now be reconstructed from (16), since bisection on the scalar trajectory provides no information about how t^* depends on θ.

The dual residual via the implicit function theorem Differentiating (16) once with respect to θ and using the chain rule gives, by the implicit function theorem,

$$\frac{\partial t^*}{\partial \theta} = -\frac{G_\theta}{G_t} = -\left.\frac{\nabla_x g_e \cdot S_\theta}{\nabla_x g_e \cdot f + \partial g_e/\partial t}\right|_*, \tag{17}$$

in which the denominator is the total time derivative $\dot{g}_e = \nabla_x g_e \cdot f + \partial g_e/\partial t$ along the trajectory and transversality of the crossing implies $\dot{g}_e \neq 0$. In dual form, (17) is realised as the quotient

$$\delta t^* = -\frac{g_e(x^-, t_*^{(0)})}{\dot{g}_e(x^-, t_*^{(0)})}, \tag{18}$$

where both g_e and $\dot{g}_e$ are evaluated on the dual-typed state x^- at the scalar grid time. The codegen-emitted partials $\nabla_x g_e$ and $\partial g_e/\partial t$ are required only for the assembly of $\dot{g}_e$; the dual chain rule propagates $\nabla_x g_e \cdot S_\theta$ through $g_e(x^-, t_*^{(0)})$ automatically. The scalar residual of the numerator (the bisection localisation tolerance) is set to zero before the division so that the value of δt^* is exactly zero by construction; only its dual derivatives remain.

The first-order dual content of (18) reproduces (17) exactly. At second order the dual quotient misses one term, because g_e is evaluated at the fixed scalar time $t_*^{(0)}$ rather than at the dual-typed time t^*: the quotient rule cannot supply the implicit G_{tt}-contribution that would arise from differentiating g_e in its time slot. Expanding (16) to second order in θ and matching coefficients identifies the missing piece. Writing $\delta t^* = \delta t^*_{(1)} + \delta t^*_{(2)} + \cdots$ and using $G(t_*^{(0)}, \theta^{(0)}) = 0$,

$$0 = G_t\, \delta t^*_{(2)} + \tfrac{1}{2} G_{tt}\, (\delta t^*_{(1)})^2 + G_{t\theta}\, \delta t^*_{(1)}\, \delta\theta + \tfrac{1}{2} G_{\theta\theta}\, \delta\theta^2,$$

of which the dual quotient supplies the $G_{t\theta}$- and $G_{\theta\theta}$-contributions but not the G_{tt}-contribution. The corrected update is therefore

$$\delta t^* \leftarrow \delta t^* - \tfrac{1}{2}\, \frac{G_{tt}}{G_t}\, (\delta t^*)^2, \tag{19}$$

where

$$G_{tt} = \left.\frac{\mathrm{d}^2}{\mathrm{d}t^2} g_e(x(t), t)\right|_*$$

is the second total time derivative of g_e along the trajectory. CppODE supplies G_{tt} either through a codegen-emitted analytic callback or, when unavailable, through a forward finite difference of $\dot{g}_e$. Equation (19) touches only the second-order dual components of δt^*; the value and first-order parts are unchanged.

Heun transport With δt^* in hand, the forward-reset-backward Heun procedure of (15) transports x^- across the event surface to the grid time $t_*^{(0)}$, with δt replaced by δt^* and the event time t_e replaced by $t_*^{(0)} + \delta t^*$. The structure of the five right-hand-side evaluations and the role of the time arguments is identical.

Simultaneous events Two or more root conditions can cross zero within the same accepted step. CppODE localises each crossing independently and groups any that share the same $t_*^{(0)}$ to within the localisation tolerance into a simultaneous-event cluster. For such a cluster the forward Heun shift is performed once to the common event surface, every triggered reset is applied at that surface (the reset maps are assumed to commute), and a single backward Heun shift returns the state to the grid time. The cost is four right-hand-side evaluations independent of the number of simultaneous events. The dual residual δt^* is computed once from the first event in the cluster with non-zero $\dot{g}_e$ at the surface; the remaining events share the same δt^*, since they all sit on the same grid time with the same parameter dependence inherited through x^-.

If every triggered event in the cluster has $\dot{g}_e = 0$ at the surface (a tangential crossing of all event functions), the denominator in (17) vanishes and the implicit function theorem fails to deliver $\partial t^*/\partial\theta$: the firing time is then insensitive to θ at first order. CppODE detects this case by the smallness of $|\dot{g}_e|$ and falls back to applying the reset as a plain map without the saltation transport, which is mathematically correct in this limit because the two saltation contributions $f^{\pm}\,(\partial t^*/\partial\theta)$ both vanish.

Connection to the saltation matrix

The first-order content of the construction above coincides with the classical saltation matrix of Filippov-type non-smooth dynamics, generalised to state resets,

$$S_\theta^+ = \frac{\partial g}{\partial x}\, S_\theta^- + \left(\frac{\partial g}{\partial t} + \frac{\partial g}{\partial x}\, f^- - f^+ \right) \frac{\partial t^*}{\partial \theta},$$

both for fixed-time events ($\partial t^*/\partial\theta = \partial t_e/\partial\theta$ supplied directly) and for root events ($\partial t^*/\partial\theta$ obtained from (17)). The implicit-function derivation chosen here extends naturally to second-order sensitivities, which the saltation-matrix form does not, and is the form realised in `cppode_integrate_times.hpp`.

6 Method selection

The recommended choice of method for the most common scenarios is summarised in Table 3.

Situation	Recommended method
Default; stiffness unknown	`"bdf"`
Stiff, frequent events	`"rb4"`
Stiff, smooth, large n_x with sparse Jacobian	`"bdf"` with `sparse = TRUE`
Non-stiff, expensive right-hand side	`"tsit5"`
Non-stiff, cheap right-hand side, long horizon, high accuracy	`"adams"`

Table 3: Recommended `method` argument for common integration scenarios. Stiffness is the dominant factor; in its absence, right-hand-side cost and required accuracy decide.

The cost of an inappropriate choice is asymmetric. An explicit method applied to a stiff problem will fail or be driven to step sizes that make the integration uneconomic, while an implicit method applied to a non-stiff problem incurs only a constant overhead per step. In the absence of prior knowledge of the spectrum of J, BDF/NDF is the safer default.

References

[1] A. C. Hindmarsh *et al.*, "SUNDIALS: Suite of nonlinear and differential/algebraic equation solvers," *ACM Transactions on Mathematical Software (TOMS)*, vol. 31, no. 3, pp. 363–396, 2005, doi: 10.1145/1089014.1089020.

[2] L. F. Shampine and M. W. Reichelt, "The MATLAB ODE suite," *SIAM Journal on Scientific Computing*, vol. 18, no. 1, pp. 1–22, 1997, doi: 10.1137/S1064827594276424.

[3] E. Hairer and G. Wanner, *Solving ordinary differential equations II: Stiff and differential-algebraic problems.* in Springer series in computational mathematics. Springer Berlin Heidelberg, 2010. Available: https://books.google.de/books?id=zZSv3acps1QC

[4] Ch. Tsitouras, "Runge–Kutta pairs of order 5(4) satisfying only the first column simplifying assumption," *Computers & Mathematics with Applications*, vol. 62, no. 2, pp. 770–775, 2011, doi: 10.1016/j.camwa.2011.06.002.

[5] E. Hairer, S. P. Nørsett, and G. Wanner, *Solving ordinary differential equations I: Nonstiff problems.* in Springer series in computational mathematics. Springer Berlin Heidelberg, 2008. Available: https://books.google.de/books?id=F93u7VcSRyYC

A BDF and NDF coefficients

Source. [2], Table 1; CppODE implementation in `cppode_multistepper.hpp`.

The Numerical Differentiation Formula (NDF) of order q modifies the classical BDFq corrector by adding a term proportional to the Nordsieck predictor error,

$$\sum_{m=1}^{q} \frac{1}{m} \nabla^m x_n \;-\; h_n\, f(x_n, t_n) \;-\; \kappa_q\, \gamma_q\, (x_n - x_n^{\mathrm{pred}}) \;=\; 0, \qquad \gamma_q \;=\; \sum_{m=1}^{q} \tfrac{1}{m}.$$

Setting $\kappa_q = 0$ recovers BDFq. The values κ_q adopted by CppODE (selectable at compile time via the argument `useNDF`, default `TRUE`) are tabulated in Table 4, together with the empirical step-size gain over classical BDFq and the resulting stability angle.

q	κ_q	Step-size gain over BDFq	Stability angle
1	-0.1850	$+26\,\%$	A-stable
2	-0.1111	$+26\,\%$	A-stable
3	-0.0823	$+26\,\%$	80°
4	-0.0415	$+12\,\%$	66°
5	0.0000	—	51° (= BDF5)

Table 4: NDF correction coefficients κ_q adopted by CppODE, the empirical step-size gain over classical BDFq at matched local error, and the resulting stability angle.

NDF5 is set to $\kappa_5 = 0$ because the BDF5 stability angle of 51° leaves no headroom for further reduction; NDF5 is therefore identical to BDF5.

B Adams-Moulton coefficients

Source. [1], ℓ-vector formulation as in CVODE; CppODE implementation in `adams_set_coefficients()` inside `cppode_multistepper.hpp`.

In contrast to BDF, the Adams-Moulton coefficients depend on the recent step-size history and are recomputed every step from the Nordsieck array. Define $\xi_j = (t_n - t_{n-j})/h_n$; the ℓ-vector recurrence reads

$$\ell_0(q) \;=\; 1, \qquad \ell_j(q) \;=\; \ell_{j-1}(q) \;+\; \xi_j^{-1}, \qquad j = 1, \dots, q,$$

followed by a polynomial expansion that yields the corrector weights $\beta_0, \dots, \beta_q$ used in the current step. Because the coefficients are step-size adaptive, no static table can reproduce them in general; the canonical fixed-step values for $h_n = h_{n-1} = \cdots$ are given in standard references [5], Table III.1.1 and reduce to the recurrence above in that limit.

The maximum order in CppODE is $q_{\max} = 12$, matching CVODE's default; the order controller may select any $q \in \{1, \dots, 12\}$ at run time.

C Rosenbrock4 coefficients

Source. [3, Sec. IV.7]; identical to the default coefficients in the `rosenbrock4` stepper of Boost.Numeric.Odeint. CppODE implementation in `default_rosenbrock_coefficients` in `cppode_rosenbrock4.hpp`.

A Rosenbrock method of s stages with diagonal coefficient γ is defined, with iteration matrix $W = (\gamma h_n)^{-1} I -$

$J(x_n, t_n)$, by the stage equations

$$W\,k_i \;=\; f\!\left(x_n + \sum_{j<i} \alpha_{ij}\,k_j,\; t_n + c_i\,h_n\right) \;+\; \frac{1}{h_n}\sum_{j<i}\frac{\gamma_{ij}}{\gamma}\,k_j \;+\; h_n\,d_i\,\frac{\partial f}{\partial t},$$

$$x_{n+1} \;=\; x_n + \sum_{i=1}^{s} m_i\,k_i, \qquad \hat{x}_{n+1} \;=\; x_n + \sum_{i=1}^{s} \hat{m}_i\,k_i.$$

The CppODE parameterisation uses $s = 6$ stages with $\gamma = 1/4$. The numerical values of all coefficients are reproduced in the following tables, indexed exactly as in the source code: the diagonal coefficient and the time partials and node positions in Table 5, the stage couplings α_{ij} and γ_{ij} in Tables 6 and 7, the sixth-row weights in Table 8, and the dense-output coefficients in Table 9.

C.1 Diagonal coefficient, time partials, and node positions

γ	0.25	d_1	0.25	c_2	0.386
d_2	-0.1043	d_3	0.1035	c_3	0.21
d_4	0.0362			c_4	0.63

Table 5: Rosenbrock4 diagonal coefficient γ, time-partial coefficients d_i, and stage node positions c_i.

C.2 Stage couplings α_{ij}

	$j=1$	$j=2$	$j=3$	$j=4$
$\alpha_{2,j}$	1.5440000000			
$\alpha_{3,j}$	0.9466785281	0.2557011699		
$\alpha_{4,j}$	3.3148251871	2.8961240160	0.9986419140	
$\alpha_{5,j}$	1.2212245092	6.0191344813	12.5370833293	−0.6878860361

Table 6: Rosenbrock4 stage-coupling coefficients α_{ij}. Row index i is the stage being formed; column index j ranges over earlier stages.

C.3 Stage couplings γ_{ij}

	$j=1$	$j=2$	$j=3$	$j=4$
$\gamma_{2,j}$	−5.6688000000			
$\gamma_{3,j}$	−2.4300933568	−0.2063599157		
$\gamma_{4,j}$	−0.1073529058	−9.5945622510	−20.4702861481	
$\gamma_{5,j}$	7.4964433140	−10.2468043146	−33.9999035282	11.7089089321

Table 7: Rosenbrock4 stage-coupling coefficients γ_{ij}, appearing in the $(1/h_n)\sum_{j<i}(\gamma_{ij}/\gamma)\,k_j$ term of the stage equation. Row and column conventions match Table 6.

C.4 Solution and error weights

The advanced solution and the embedded error estimate are formed from the stage values $k_1, \ldots, k_5$ via

$$x_{n+1} \;=\; x_n + \sum_{i=1}^{5} \alpha_{6,i}\,k_i, \qquad e_{n+1} \;=\; \sum_{i=1}^{5} \gamma_{6,i}\,k_i,$$

with the sixth-row coefficients listed in Table 8.

	$i=1$	$i=2$	$i=3$	$i=4$	$i=5$
$\alpha_{6,i}$	1.2212245092	6.0191344813	12.5370833293	−0.6878860361	1.0
$\gamma_{6,i}$	8.0832467959	−7.9811329881	−31.5215943287	16.3193054312	−6.0588182388

Table 8: Rosenbrock4 sixth-row coefficients. The $\alpha_{6,i}$ values combine the stage values into the advanced solution x_{n+1}; the $\gamma_{6,i}$ values supply the embedded order-3 error estimate e_{n+1}.

C.5 Dense-output coefficients

Continuous output on $[t_n, t_{n+1}]$ is constructed from the five intermediate stage values together with two auxiliary coefficient sets $d_i^{(2)}$ and $d_i^{(3)}$, listed in Table 9, that combine the stages with the step solution to produce an order-3 interpolant.

	$i=1$	$i=2$	$i=3$	$i=4$	$i=5$
$d_i^{(2)}$	10.1262350834	−7.4879958776	−34.8009186156	−7.9927717076	1.0251377233
$d_i^{(3)}$	−0.6762803393	6.0877146517	16.4308432089	24.7672251142	−6.5943891257

Table 9: Rosenbrock4 dense-output coefficients $d_i^{(2)}$ and $d_i^{(3)}$. Together with the stage values they form the order-3 interpolant on $[t_n, t_{n+1}]$ used to serve user output points falling between accepted steps.

D Tsitouras 5(4) coefficients

Source. [4], Table 1 and equations (7)–(8); CppODE implementation in `cppode_tsit5.hpp`.

The Butcher tableau of the Tsitouras 5(4) pair, reproduced in Table 10, has $s = 7$ stages and obeys the simplifying assumption $a_{i,1} = 0$ for $i \geq 2$; the seventh stage is the FSAL stage. Both rows of weights are listed beneath the A matrix: b_i for the order-5 solution and $\hat{b}_i$ for the embedded order-4 estimate.

0							
0.161	0.161						
0.327	−0.0084806555	0.3354806555					
0.9	2.8971530571	−6.3594484900	4.3622954329				
0.9800255409	5.3258648284	−11.7488835641	7.4955393429	−0.0924950664			
1.0	5.8614554429	−12.9209693178	8.1593678986	−0.0715849733	−0.0282690504		
1.0	0.0964607668	0.01	0.4798896504	1.3790085741	−3.2900695154	2.3247105241	
b_i	0.0964607668	0.01	0.4798896504	1.3790085741	−3.2900695154	2.3247105241	0
$\hat{b}_i$	0.0946807557	0.0091835655	0.4877705284	1.2342975669	−2.7077123499	1.8666284182	0.0151515152

Table 10: Butcher tableau of the Tsitouras 5(4) pair. The first column lists the node positions c_i; the centre block is the strictly lower-triangular coefficient matrix a_{ij}. The b_i row gives the order-5 weights, and the $\hat{b}_i$ row gives the embedded order-4 weights.

The seventh stage is $k_7 = f(x_{n+1}, t_n + h_n)$, which is re-used as k_1 at step $n+1$. Because $b_7 = 0$, the order-5 solution does not depend on k_7; because $\hat{b}_7 \neq 0$, the embedded order-4 estimate does. The local-error estimator used by the step controller is

$$e_n \;=\; h_n \sum_{i=1}^{7} (\hat{b}_i - b_i)\, k_i .$$

The seven differences $\hat{b}_i - b_i$ are stored in `cppode_tsit5.hpp` as the coefficients `e1`, ..., `e7` and reproduced in Table 11.

i	$\hat{b}_i - b_i$
1	−0.0017800111
2	−0.0008164345
3	0.0078808780
4	−0.1447110072
5	0.5823571655
6	−0.4580821059
7	0.0151515152

Table 11: Tsit5 embedded error coefficients $\hat{b}_i - b_i$, stored in `cppode_tsit5.hpp` as `e1`, ..., `e7`. The seven values sum to numerical zero, the consistency condition for the embedded pair.

The seven values sum to numerical zero, as required for the consistency of the embedded pair, and the row $a_{7,j}$ of the A matrix coincides with b_j for $j = 1, \ldots, 6$ (the FSAL property). Continuous output on $[t_n, t_{n+1}]$ is delivered by the cubic Hermite interpolant on the accepted-step endpoints (x_n, x_{n+1}) and their derivatives $(f_n, f_{n+1}) = (k_1, k_7)$. Both endpoint derivatives are already in cache from the FSAL property, so no additional right-hand-side evaluations are required for output points.